E1-small pre-registration — frozen snapshot

docs/osf-prereg-e1-small-2026-10-25.md at commit 82dc277df209 (file SHA-256 6d47d8b7…93208463)

E1-small: replay controls at 4B (dense) pre-registration

Status: DRAFT — not yet filed on OSF (to be filed by the PI). **Date:** 2026-10-25 (revision 2: PI decisions of 2026-10-25 19:50 and 20:06, 2026-10-26 22:09, 2026-10-27 — model pin, own freeze, kernels (dropped), raw tokenizer, self-contained text, macM1 pipeline smoke, determinism, allocator; §10). **Origin:** PI decision of 2026-10-25 (14:43): run E1 first at small scale on a Qwen 4B dense model on KXKM-AI (one RTX 4090, 24 GB) with the PyTorch / CUDA harness, as a **separate** pre-registration. The 35B study (`docs/osf-prereg-e1-replay-controls-2026-10-23.md`, "the E1 draft") continues later and is not changed by this one. **Results are never pooled across model sizes, studies or instruments** (§0, §8). **Self-contained (PI decision S7).** §1-§8 reproduce the protocol of revision 9 of the E1 draft (commit 9172768, SHA-256 `ae00aea5…bf71b5`), edited only where the model size or the instrument forces it; §11 lists every such edit. Later revisions of the E1 draft do not change this document. **Parent OSF:** 10.17605/OSF.IO/Q6JYN **Parent backlog item:** `EXPERIMENT-BACKLOG.md` E1. **Prior evidence (not modified):** `docs/milestones/pilot-cycle3-real-1p5b-CORRECTION-2026-06-19.md`. It invalidates the 1.5B GO and the 1.5B↔7B scaling-law GO (mega_v2 fixture leak). It leaves a replay-specific residual `p_min − p_ctrl_sft ≈ +0.10` (N = 20, 95 % CI [+0.085, +0.109]) as a non-adjudicable candidate, confounded by template / format transfer. **c_version at drafting:** C-v0.25.0+PARTIAL **Substrate:** Qwen3.5-4B (dense), bf16, `Qwen/Qwen3.5-4B` at revision `851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a` (PI decision S1, §9.1). **Instrument (registered):** PyTorch 2.11.0 + HF transformers 5.7.0 on one NVIDIA RTX 4090 (KXKM-AI), every parameter on that device, linear attention through the transformers torch fallback (S6: the fla / causal-conv1d kernels were evaluated and dropped by the pre-fixed fp32 diagnostic rule, §9.2), token ids from the raw pinned `tokenizer.json` (S8), deterministic mode (S10, §9.7c) and the `expandable_segments:True` CUDA allocator (S11); §9.2. **Harness:** `scripts/e1_harness.py` with `HL_STUDY=e1-small`, PyTorch backend `scripts/pilot_cycle3_cuda.py`, runner `scripts/run_e1_small_cuda.sh`; analysis `scripts/analyze_e1_cuda.py`. **Study gate:** as the E1 draft: E1-small does not run until the independent fact corpus of §5.1 is frozen — for E1-small with the 4B headroom (S2, §9.3). The fixtures `tests/fixtures/mmlu_real.jsonl` (114 rows, SHA-256 `4d881827…a09dd`) and `tests/fixtures/hellaswag_sanity.jsonl` (200 rows, SHA-256 `2ab581dc…0a979`) are a **pilot / smoke set only** (§5.3). **Compute:** 141 confirmatory seeds × 10 profiles = 1,410 cells, plus 120 tuning cells, 60 LR-matching calibration cells and 30 smoke cells: **1,620 cells** (§5, §9). Without the exploratory arm: 1,416. **Power script:** `scripts/prereg_power_2026_10.py` (§3). **Lock commit:** *(filled by introducing commit hash at filing)* **Lock timestamp:** *(filled at filing, pre-run)*

§0 Scientific role and scope

E1-small is a clean re-test, at small scale, of the evidence that the 2026-06-19 correction invalidated: the 1.5B framework-C GO and the 1.5B↔7B scaling-law GO (mega_v2 fixture leak), and the leak-free residual `p_min − p_ctrl_sft ≈ +0.10` that the correction left as a non-adjudicable candidate confounded by template / format transfer. It applies the E1 design (matched-compute controls, prompt-only control, tuned vanilla replay, restricted MC on-policy replay, the independent fact corpus) to a model in the size range of that evidence (1.5B-7B), on the hardware that produced it (KXKM-AI, RTX 4090).

What it can establish. The states of H14-A, H14-B and H14-C (§2) for Qwen3.5-4B on the frozen fact corpus, under the E1 protocol.

What it cannot establish. Nothing about the 35B model or any other size. A result here does not predict, replace,

or pre-empt the 35B study; the 35B study's hypotheses, N and α are untouched by it, and neither study's data enter the other's analysis. No scaling claim (4B vs 35B) is pre-registered; any comparison of the two studies is descriptive and outside both confirmatory families. E1-small also does not adjudicate the historical mega_v2 residual (different task, endpoint and model), exactly as the E1 draft §1 states for 35B.

§1 Background

The 2026-06-19 correction established two design rules for any LLM- tier framework-C claim:

1. score the effect as replay **minus a matched control**, not post − pre;
2. control for template / format transfer, because row-disjoint items that share a template still transfer.

Two 2026 results raise the bar further:

- Chen et al. (2026), *On-Policy Replay for Continual Supervised Fine-Tuning* (arXiv:2605.29495). Replaying the current checkpoint's own reward-filtered **generated** responses on past prompts cuts forgetting by 42-46 % versus **tuned vanilla replay** on TRACE (7-8B). Low-score on-policy replay is worse than vanilla.
- Wang et al. (2026), *Denser ≠ Better: Limits of On-Policy Self-Distillation for Continual Post-Training* (arXiv:2607.01763). On-policy self-distillation can **collapse**, with drift amplified by the teacher–student loop. Any self-generated replay arm needs a pre-specified drift / collapse diagnostic (§6).

Three further 2026 works frame the design:

- Behrouz, Hashemi, Javanmard & Mirrokni, *Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories* (arXiv:2606.03979; also an ICLR 2027 submission, OpenReview `4BIMcGrZjq`). A "Sleep" paradigm with (1) memory consolidation by upward distillation (Knowledge Seeding: on-policy distillation combined with RL-based imitation learning) and (2) "Dreaming", an RL-driven generation of synthetic curricula, evaluated on long-horizon, continual-learning, knowledge-incorporation and few-shot tasks. **Differentiation:** that work proposes a sleep/dream method; E1 does not. E1 is a controlled, token-matched comparison at one fixed 4B dense checkpoint of the existing GENIAL dream arm (`p_equ`) against tuned vanilla replay, restricted on-policy replay and a same-checkpoint prompt-only control, with the DOWNSCALE contribution isolated by ablation (H14-B). No claim here is novel because of the words "sleep" or "dream", and E1 does not reproduce or test the Behrouz et al. method.
- Marek, Cho, Qiu, Chunara, Izmailov & Wilson, *Forgetting in Language Models: Capacity, Optimization, and Self-Generated Replay* (arXiv:2605.26097). Self-generated samples serve as effective replay data, nearly eliminating forgetting; low learning rates reduce forgetting but need substantially more training steps, and replay breaks that trade-off. Learning rate and step count therefore change the outcome directly, which is why the controls are tuned (§5), and it motivates the DOWNSCALE caveat in §7.
- Peng, Li, Ouyang, Yuan & Ding, *Diagnosing and Mitigating Thinking Collapse in On-Policy Self-Distillation* (arXiv:2607.10805), reports a collapse of native intermediate reasoning under on-policy self-distillation and a mitigation that anchors to a reference derived from the frozen base model. It motivates the frozen-reference diagnostic of §6. A second report of collapse, in latent on-policy distillation (LastOPD, arXiv:2609.28845), is consistent with it; no newer result removes the need for the diagnostic.

In this harness `p_min` **is** SGD replay at default hyperparameters (`HL_LR = 1e−4`, `HL_INNER = 5`): vanilla, untuned replay. The framework-specific part is what `p_equ` (replay + DOWNSCALE) and `p_max` (+ RESTRUCTURE) add. RECOMBINE is a weight-neutral no-op in the CUDA harness.

What "content" means here. Replay items are row-disjoint from the items each seed is scored on (`_cell_split`), and in this revision also come from the same item-disjoint partition (§5.2). Any eval gain from replay is therefore transfer from **other** items: it can come from answer-format adaptation, domain text, or learning to emit correct answers in general. Because the corpus items are independent facts (§5.1), replaying one item does not rehearse the fact scored by another. H14-A compares answer-containing replay with prompt-only replay. It does **not** separate answer format from answer content (the prompt-only control sees no completions at all), and it does not test recall of the scored facts.

Scope. All conclusions are about this experiment: Qwen3.5-4B, on the registered instrument, on the E1-small FROZEN fact corpus of §5.1; nothing is inferred about the 35B model of the E1 draft (§0). The 2026-06-19 residual was measured on hand-authored mega_v2 facts with a 1.5B model and a different endpoint. This study motivates, but cannot adjudicate, the mechanism of that historical residual.

§2 Hypotheses (confirmatory)

Unit and pairing. Unit = seed. Within a seed every profile uses the same replay / eval split, the same eval items for pre and post (except p_noise, whose post-evaluation uses the different draw _cell_split(seed + 9973) by design, to measure item-sampling noise; p_noise enters no confirmatory contrast), and starts from the same restored base weights. All contrasts are **paired by seed**: for contrast (X, Y) the analysed quantity is D_s = Δ_s(X) − Δ_s(Y), with Δ = post − pre on acc (Endpoint, below).

Endpoint. acc = accuracy on that seed's n_eval eval items of the confirmatory partition of the fact corpus (§5.1; n_eval ≥ 130, fixed by the rule of §5.2). This replaces the previous draft's MMLU / HellaSwag composite; "accuracy" below always means acc.

SESOI and margins. δ = **0.01** accuracy points (PI decision of 2026-10-23, half the previous draft's 0.02; one tenth of the 1.5B residual of +0.10; 1.3 items at n_eval = 130). It is a fixed raw margin used as the superiority floor, the bound for "no effect of practical size", and the non-inferiority margin. It is not rescaled by any observed SD. For reference, it equals the old 0.29-SD convention only if SD_D = 0.0345 (§3).

Paired procedures (exact). For a contrast with retained n: $\bar{D}$ = mean(D), s = sd(D, ddof=1), se = s / √n, q = $t_{1-\alpha, n-1}$, L = $\bar{D}$ − q·se, U = $\bar{D}$ + q·se.

- Superiority: reject iff t = $\bar{D}$ / se > q (ttest_rel(X, Y, alternative="greater")), i.e. L > 0.
- Non-inferiority (margin δ): reject iff ($\bar{D}$ + δ) / se > q, i.e. L > −δ.
- Paired TOST (bounds ±δ): equivalence iff L > −δ and U < δ.
- **Degenerate case.** If every retained D is identical (sd(D) = 0), the t statistic is undefined; then L = U = mean(D) and the state follows from these bounds. g_z is then reported as undefined.
- Effect sizes reported: raw D with the (1 − 2α) interval [L, U], and g_z = $\bar{D}$ / s × (1 − 3/(4(n − 1) − 1)).

kiki_oniric.eval.statistics.tost_equivalence is **not** used: it computes an independent-samples standard error with df = $n_1 + n_2 - 2$, which is wrong for paired data. The paired procedures above are implemented as kiki_oniric.eval.statistics.paired_tost and paired_noninferiority (PR #42; tested against hand-computed examples and SciPy one-sided p-values, identical differences handled exactly), which closes task 1 of §9.

Family. Three hypotheses, Bonferroni α = 0.05 / 3 = **1/60 ≈ 0.0167** (same α as the E1 harness).

- **H14-A (answer-containing vs prompt-only replay)** — p_min vs p_ctrl_template (prompt-only control, §5). Primary claim: superiority.
- **H14-B (DOWNSCALE on top of tuned replay)** — p_equ vs p_ctrl_vanilla_replay_tuned. Both use the **same** tuned replay configuration and the identical update sequence (§5); the only difference is the DOWNSCALE step after replay. Primary claim: superiority.
- **H14-C (non-inferiority to on-policy replay)** — p_equ vs p_ctrl_onpolicy_replay. Primary claim: non-inferiority at δ.

Per-hypothesis states (evaluated in order; first match; the list is exhaustive):

For H14-A and H14-B:

State	Rule
UNINTERPRETABLE	a validity gate of §4 / §6 fails for this contrast
SUPERIOR	L > 0 and $\bar{D}$ ≥ δ
POSITIVE-BELOW-SESOI	L > 0 and $\bar{D}$ < δ
BOUNDED-NO-GAIN	U < δ (gain below δ; includes negative estimates)
INCONCLUSIVE	none of the above

For H14-C:

State	Rule
UNINTERPRETABLE	a validity gate of §4 / §6 fails for this contrast

State	Rule
NON-INFERIOR	$L > -\delta$
INFERIOR (exploratory)	$U < -\delta$ (worse than OPR by more than δ)
INCONCLUSIVE	neither

What SUPERIOR means. $L > 0$ establishes a positive population difference at level α; $\bar{D} \geq \delta$ filters the **point estimate**. SUPERIOR reads "a positive effect whose estimate is at least δ", never "the population effect is at least δ".

Error accounting. Each hypothesis spends α once. For H14-A/B the confirmatory conclusions are SUPERIOR (false only if the true $\mu \leq 0$) and BOUNDED-NO-GAIN (false only if $\mu \geq \delta$); as $\delta > 0$, at most one can be false for any μ. For H14-C only NON-INFERIOR is confirmatory; INFERIOR is exploratory and spends no α (both can be false at $\mu = -\delta$ exactly). The family bound is $3\alpha = 0.05$.

$U < 0$ in H14-A/B, and the paired TOST in H14-C, are reported as secondary results; no harm or equivalence conclusion is part of the confirmatory family. A non-significant superiority test never establishes absence: only BOUNDED-NO-GAIN does (and only below δ). Failing to show non-inferiority is INCONCLUSIVE, not inferiority.

Secondary (reported, outside the family, nominal $\alpha = 0.05$, same state rules): every dream profile (`p_min`, `p_equ`, `p_max`) against every control, which keeps the `analyze_e1_cuda.py` defensibility signature `dream > p_ctrl_sft > p_ctrl_shuffle ≈ p_noise`.

§3 Power analysis

n = 141 paired seeds; reproduced by `scripts/prereg_power_2026_10.py` (sections 1, 7 and 8; superiority rules by sufficient-statistic Monte Carlo, 200 000 draws; one-sided bound rules at $\mu = 0$ exact).

- Test-only superiority MDE at 80 % power, one-sided $\alpha = 1/60$: **$d_z = 0.252$** ($g_z \approx 0.251$).
- Standardised check (margin fixed at 0.29 population SD, true difference 0): NI ≈ **0.901**, paired TOST ≈ **0.802** (≈ 80 %). This is where N = 141 comes from (backlog E1).

The confirmatory margins are raw ($\delta = 0.01$), so power depends on the unknown SD of the paired difference, SD_D:

SD_D	δ/SD_D	SUPERIOR at μ = 0.01	μ = 0.015	μ = 0.02	BOUNDED-NO-GAIN / NI at μ = 0	TOST at μ = 0
0.015	0.67	0.50	1.00	1.00	1.00	1.00
0.02	0.50	0.50	1.00	1.00	1.00	1.00
0.028	0.36	0.50	0.98	1.00	0.98	0.96
0.035	0.29	0.50	0.96	1.00	0.89	0.78
0.05	0.20	0.50	0.88	0.99	0.59	0.18
0.069	0.14	0.33	0.67	0.90	0.34	0.00

The complete SUPERIOR rule has ≈ 0.50 power when the true effect equals δ (the observed-mean floor) whenever the test itself is well powered; at SD_D = 0.069 the test is the binding constraint (0.33).

Planning SD. The only measured SD of a paired replay-minus-control difference in this harness family is **SD_D = 0.028** (Run A of the 2026-06-19 correction: `p_min − p_ctrl_sft`, Qwen2.5-1.5B, 20 seeds, hand-authored facts, 10 eval items). It is the planning value. It comes from another model, endpoint and eval size, so it is a rough anchor, not evidence for this design.

What N = 141 can and cannot establish (stated plainly). At the planning SD_D = 0.028, N = 141 gives 0.98 for BOUNDED-NO-GAIN / NI and 0.96 for the secondary TOST; 72 (BOUNDED / NI) and 94 (TOST) seeds would suffice. N = 141 reaches 80 % only if SD_D ≤ 0.0397 (BOUNDED / NI) or ≤ 0.0345 (TOST). **If SD_D is larger, the registered N is insufficient** for an 80 %-powered "no practical effect" conclusion at δ = 0.01: at SD_D = 0.05 it needs 223 / 294 seeds, and at the previous draft's reference SD_D = 0.069 it needs 423 / 557. The corpus-size rule of §5.2 is set so that item sampling alone uses at most half of the 0.0397 budget.

No prior run reports SD_D for these contrasts at 4B. The pre-confirmatory proxy is the 10-seed tuning data: the SD over tuning seeds of the paired difference between the two selected configurations (vanilla vs OPR) on the tuning partition. It is a rough, item-shifted proxy for the H14-C SD_D and none exists for H14-A/B. It is recorded in the frozen tuning amendment for information only. **N stays 141 whatever the proxy shows**; if the proxy exceeds 0.0397, the amendment states the correspondingly reduced BOUNDED / NI power before any confirmatory seed runs.

§4 Exclusions, replacement and retained-N inference

- **Cell-level.** A cell is excluded if a BLOCKING invariant is violated, or pre/post scores are non-finite. A contrast uses the seeds where **both** of its cells are retained (pairwise deletion).
- **No replacement.** Confirmatory seeds are exactly 0-140. Excluded seeds are not replaced; inference uses the retained n, and every exclusion is reported with its reason.
- **OPR feasibility.** If the pre-replay checkpoint answers **zero** of a seed's 40 replay prompts correctly, the OPR corpus is empty. This is a **feasibility** failure determined before any treatment is applied, not a selective removal of hard seeds: that seed is dropped from H14-C only, and the count is reported. If more than 7 seeds (> 5 %) are dropped this way, H14-C is UNINTERPRETABLE. The distribution of OPR corpus sizes (1-40 items) is reported.
- **Differential-exclusion gate.** If a confirmatory contrast loses more than 7 of 141 seeds to cell-level exclusions, its state is UNINTERPRETABLE. Below that, the estimand is the mean paired difference over seeds whose two cells both complete, and the error rates of §2 are conditional on that set; the gate limits, but does not remove, selection by treatment-dependent failures.
- **Incomplete run.** If compute ends before 141 seeds are complete, **all** H14 hypotheses become exploratory (§8).

§5 Corpus, partitions, profiles and matched compute

§5.1 Fact corpus (study gate)

The confirmatory and tuning items come from a new corpus that must exist, frozen and hashed, before tuning starts and before filing. Required properties:

1. **Format.** Four-choice items (`question`, `choices[4]`, `answer` index), scored by `_score_choice` exactly like the pilot MMLU rows. One gold answer per item; the three distractors are of the same type as the answer. Question plus gold answer is at most 256 tokens under the pinned tokenizer, which keeps the ≤ 512-token memory assumption of §9.
2. **Independence.** One fact per item. Items are written or sourced individually, never produced by a fill-in template generator. Masking named entities, numbers and dates in the question gives a *stem skeleton*; no skeleton may be shared by more than 5 items. Items that state, paraphrase or presuppose the same fact are one *fact group*.
3. **Deduplication.** Exact duplicates (after Unicode NFKC, lower-casing and whitespace collapse of question + gold answer) are removed. Near-duplicates (character 5-gram Jaccard ≥ 0.6 on the normalised question, or an identical gold answer string with Jaccard ≥ 0.4) are merged into one fact group (connected components, §5.1b). Each fact group is then reduced to one item (the one with the lowest source-item hash `src_hash`, §5.1b), so every retained item is its own fact group and the row-level `_cell_split` is fact-disjoint within a seed. Any item that is a near-duplicate of a row of `mmlu_real.jsonl`, `hellaswag_sanity.jsonl` or the 20 hand-authored mega_v2 facts is removed.
4. **Provenance.** Each item records its source (citation, DOI or URL, or "authored"), author, licence and creation date. The build script and its inputs are versioned in this repository. **PI decision 2026-10-25:** citation, author and licence are recorded per source; each manifest item carries its source, `source_id` and a `provenance_date`: the item's own date for the dated sources (`provenance_date_kind = item`) and, for the undated `mmlu_cf` items, the release date of the pinned MMLU-CF revision `c25b89a` (2025-01-08), labelled `source_release`, i.e. a source date, not an item creation date.
5. **Headroom.** Before freezing, the base checkpoint is scored on the tuning partition only. If its accuracy there exceeds 0.80, the corpus is not used (too little room for a gain of δ) and is extended or replaced before filing. Confirmatory items are never scored before the first confirmatory seed.

6. **Size.** After deduplication and reduction: confirmatory partition ≥ **300** items and tuning partition ≥ **120** items. The confirmatory minimum is `n_replay + n_eval` = 40 + 255 = 295, rounded up to 300, where 255 is the `n_eval` the §5.2 rule requires for a discordance of 0.20; the tuning minimum is 40 replay + 80 eval items.
7. **Partitioning.** Fact groups (never single items) are assigned to the confirmatory or tuning partition by SHA-256 of a content-blind group key, in the manner of `scripts/e1_fixture_partition.py`; no difficulty or model output is used. A fact group never straddles the partitions, so the two partitions are item- and fact-disjoint.
8. **Hashing.** Per-item SHA-256 of the canonical JSON line (the final hash `item_sha256` of §5.1b; the pre-permutation `src_hash` is stored in the item and drives every random choice), the corpus file SHA-256, per-partition hashes (SHA-256 of sorted item hashes) and a manifest with a `--check` mode, as for the pilot manifest. The corpus and partition hashes are written into the filed version; any later change to the corpus is a deviation.

§5.1a Sources (PI decision 2026-10-24: all three public sets)

The PI chose to use all three proposed public fact sets and confirmed on 2026-10-24 (17:25) the design: one pooled item bank, with `source` recorded on every item and pre-registered as a stratum; partitions are stratified by source; per-source results are reported descriptively; the primary analysis (§2) is pooled. The §5.1 properties apply to every item, whatever its source.

`source`	Set and pin	Licence	Role
`realtimeqa`	RealTime QA, `github.com/realtimeqa/realtimeqa_public` at commit `e288dcfe1d7818bf48299249ae892939637775db` (2026-10-23), files `past/2026/*_qa.jsonl`; per-file SHA-256 computed and pinned at build	none declared (questions from CNN / The Week news quizzes)	human-written, native 4-choice, dated news items
`daily_oracle`	Daily Oracle, HF `agentic-learning-ai-lab/daily-oracle` at revision `455b35b2a2415034b05ef91e56c80191fef4a737`, `mc_questions_2020-01-01_2026-07-18.csv` (119 302 323 B, LFS SHA-256 `c8c0b5adaa266fa5e9c1ec59b9cd7d9182c89607d39eb0ef2b269cbb12e58178`)	CC BY 4.0	LLM-generated, 4-choice, dated, one article per item
`mmlu_cf`	MMLU-CF validation, HF `microsoft/MMLU-CF` at revision `c25b89a968a2062e422dd96ddf0fe3387507f0dd`, `val.csv` (2 247 391 B, SHA-256 `28816045197decd701b1a0e51923b43684d708d8c06ec62d20541f90939ac49f`; `val.parquet` LFS `a44a6bb1…77173d`)	CDLA-Permissive-2.0	decontaminated MMLU-style items, undated

Rules fixed for all sources (applied by the task-7 build script before the §5.1 deduplication, and recorded per item):

1. **Date cutoff.** Dated items (`realtimeqa` `question_date`, `daily_oracle` `date`) are kept only if dated **2026-04-16 or later** (the cutoff of the shared corpus, set after the Qwen3.6-35B-A3B release of 2026-04-15; Qwen3.5-4B was released 2026-02-27), so they post-date the model. `mmlu_cf` items carry no date: they are kept with a contamination caveat (the validation set is public since December 2024) and rely on the headroom rule (5).
2. **Whitespace normalisation.** Question and each option are stripped of leading / trailing whitespace before hashing and prompting (in Daily Oracle the gold option never carries trailing spaces while distractors do, a format leak). After normalisation the build reports, per source, how often the gold option is the longest; this is descriptive.
3. **Same-type distractors.** `mmlu_cf` items with a "none of the above"-type option are removed (§5.1, property 1). **Widened by the PI on 2026-10-25 to a frozen exact list** (`NOTA_OPTIONS` in `scripts/e1_sources.`

py). An item is removed iff one of its options, after rule 2 (leading / trailing whitespace stripped) and with no other normalisation, equals one of these 27 strings exactly (case-sensitive, no fuzzy match): `None of the other choices`; `None of the mentioned`, `None of the Mentioned`, `none of the mentioned`, `None of the above mentioned`, `none of the above mentioned`; `None of the above`, `None of the above.`, `None of the Above.`, `none of the above`, `none of the above.`, `None of above`; `None of these`, `None of These`, `None of these.`, `none of these`, `none of these.`; `None of the choices`, `none of the choices.`, `none of the options`; `None of the above is correct.`, `None of the above answers is correct.`, `None of the above are true`, `None of these alternatives is correct.`, `None of these answer choices are correct.`, `None of these answers is correct.`; `None is applicable`. The list was enumerated from every distinct option of the pinned `val.csv` that starts with "none" (case-insensitive) and keeps exactly the options that only refer to the other options. Not on the list (kept): bare `None` / `none` / `NONE` / `None.` (can be a substantive answer, e.g. a count) and options that add content (`None of the statements are true`, `None of these (Socrates)`, `None of the above is a difference.`, `None of the above because the temporal method is not allowed under FASB ASC 830.`, `None of the above accurately describe financial reports pre-1900.`, `None of the production and service departments`, `none of the major theories`). "All of the above" and "Both …" options are not covered. The rule applies to `mmlu_cf` only; matches in the dated sources are counted in the build report, not removed. The build reports, per variant, the number of items carrying it.

4. **Date injection.** Dated items are prompted as `"Date: YYYY-MM-DD\n" + question` with their own date; undated items get no prefix. (Many news items say "this week".)
5. **Headroom per source.** The §5.1 headroom check (base accuracy ≤ 0.80 on the tuning partition) is applied to the pooled tuning partition **and to each source**; a source above 0.80 is dropped before freezing and the drop is recorded.
6. **Option order.** Choices are permuted per item with `default_rng(int(src_hash[:16], 16)).permutation(4)`, where `src_hash` is the **source-item hash** of §5.1b (computed before any permutation), and the gold index is updated; the permutation is part of the frozen item, whose **final hash** is computed after it (no circularity).
7. **Balance (confirmed by the PI on 2026-10-24).** The dated sources give about 545 (`realtimeqa`) and 492 (`daily_oracle`) items after the cutoff, while `mmlu_cf` has about 5 400 after rule 3. `mmlu_cf` is capped at the size of the largest dated source, by the exact algorithm of §5.1b. The confirmatory (≥ 300) and tuning (≥ 120) minima of §5.1 apply to the pool.
8. **Stratified partition.** Fixed per-source quotas (§5.1b), so each source contributes to the confirmatory and tuning partitions in the same proportion.
9. **Licences.** `realtimeqa` text is not redistributed: the repository and any publication carry only item IDs (source file, line, date), item SHA-256 and the build script; the item text is rebuilt from the pinned commit. `daily_oracle` (CC BY 4.0) and `mmlu_cf` (CDLA-Permissive-2.0) are attributed in the milestone.
10. **Reporting.** Every §2 statistic is also reported per source (descriptive, no state); the confirmatory states are pooled.

Only the pins and metadata above were fetched (the 2.2 MB MMLU-CF `val.csv` to compute its SHA-256, then deleted); the corpus is built by task 7 once the study gate is passed.

§5.1b Construction (exact; independent review round 4)

Two hashes per item, so randomisation never depends on its own output:

- **Source-item hash** `src_hash` = SHA-256 of the canonical JSON (UTF-8, `sort_keys=True`, `separators=(",", ":")`, `ensure_ascii=False`) of `{source, source_id, date, question, choices, answer}` **after normalisation and before permutation** (`choices` in source order, `answer` the source index; `date` `YYYY-MM-DD` or `null`; `source_id` is a string: for `realtimeqa` the repository-relative path from the repository root with forward slashes, a colon and the 0-based line index, e.g. `past/2026/20260501_qa.jsonl:3`; for `daily_oracle` the 0-based data-row index of the pinned CSV as a decimal string, e.g. 1527; for `mmlu_cf` `val.csv:` and the 0-based data-row index, e.g. `val.csv:42`). It drives every random or ordered choice below, including the fact-group representative of §5.1 property 3.
- **Final hash** `item_sha256` = SHA-256 of the canonical JSON of the frozen item `{source, source_id, date, prompt, choices, answer, src_hash}` **after** permutation (`prompt` includes the date prefix). It is the §5.1 per-item hash used in the manifest and the partition hashes.

Construction order (each step logs its in / out counts per source):

```
1  load      verify every pin (§5.1a table); map rows to source items
2  normalise strip question and options (rule 2); NFKC + lower-case +
             whitespace collapse only inside dedup / skeleton keys
3  exclude   drop: != 4 choices, empty field, invalid gold, a
             NOTA_OPTIONS option in mmlu_cf (rule 3, exact list),
             question + gold > 256 tokens (pinned tokenizer: question
             and gold encoded separately, no special tokens, summed)
4  date      dated sources: keep date >= 2026-04-16 (rule 1); a kept
             item that is a near-duplicate (§5.1 property 3) of any
             dropped pre-cutoff item of the same source is dropped
             too (recurring quiz questions)
5  dedupe    §5.1 property 3 over the pooled survivors and against
             the pilot fixtures / mega_v2 facts; each fact group keeps
             its lowest src_hash; skeletons with > 5 items keep the 5
             lowest src_hash
6  cap       n_rt, n_do, n_cf = post-dedup counts; k = max(n_rt, n_do);
             mmlu_cf: sort by src_hash ascending, keep the first
             min(n_cf, k)
7  split     per source s, items sorted by src_hash ascending;
             t_s = ceil(0.30 * n_s) lowest go to TUNING, the other
             n_s - t_s to CONFIRMATORY (each item is its own fact
             group after step 5, so the split is fact-disjoint)
8  permute   rule 6 with src_hash; add the date prefix (rule 4);
             compute item_sha256; write corpus, manifest, hashes
9  headroom  score the base checkpoint on TUNING only (§5.1
             property 5), pooled and per source
```

Dedupe (5) comes after the date filter (4) so that an item removed for its date can never knock out a kept duplicate; step 4's second clause covers contamination through such duplicates. Its reference set is **every pre-cutoff row of the same source after step 2, including rows that step 3 would exclude** (a malformed earlier copy still leaks the fact).

Exact definitions (reference implementation `scripts/e1_corpus_core.py`, unit-tested on synthetic items).

- *Keys.* `norm_key(t)` = NFKC, lower-case, whitespace collapsed to single spaces, trimmed. `grams(t)` = the set of character 5-grams of `norm_key(t)`; a non-empty key shorter than 5 characters is one gram; an empty key gives the empty set. `J(a, b)` = |a ∩ b| / |a ∪ b|, and 0 when both are empty (never a duplicate).
- *Near-duplicate relation.* Items x, y are near-duplicates iff `J(grams(q_x), grams(q_y)) ≥ 0.6`, or `J ≥ 0.4` and their normalised gold options are equal; the gold is consulted only once `J ≥ 0.4`. *Invalid gold (final review):* a row whose `choices` is not a list or whose `answer` is not an integer index into it (an empty list, an out-of-range or negative index, a non-integer) has no gold. This can only happen among the pre-cutoff reference rows, which keep rows that step 3 would exclude; such a row matches by the question rule (`J ≥ 0.6`) only, never by the equal-gold rule. Keys and gram sets are computed once per row.
- *Grouping.* Union-find over **all pairs** of the step-5 survivors plus the reference rows (pilot fixtures, mega_v2 facts): every near-duplicate pair is joined. A connected component that contains a reference row is removed entirely; every other component keeps its item with the lowest `src_hash`. Step 4's contamination check uses the same relation against the pre-cutoff reference set.
- *Skeleton* (dependency-free; after the grouping). On the NFKC question, in this order: (i) ISO dates `\b\d{4}-\d{2}-\d{2}\b` and month-name dates (`Jan`/`January` … `Dec`/`December`, optional `.`, a 1-2 digit day with optional `st|nd|rd|th`, optional `,` YYYY) → <d>; (ii) numbers `\d(?:[\d,.]*\d)?` → <n>; (iii) the first character is lower-cased (sentence-initial word); (iv) runs of capitalised tokens `[A-Z][\w'-]*(?:\s+[A-Z][\w'-]*)*` → <e>; (v) lower-case and whitespace collapse. Example: "What did Emmanuel Macron announce on May 3, 2026 about 1,200 New Jobs?" → `what did <e> announce on <d> about <n> <e>?`. Items sharing a skeleton beyond 5 keep the 5 with the lowest `src_hash`.

Headroom drop and rebalancing (`headroom_rebalance`). Every tuning item must have been scored. Outcomes are ACCEPT or **REJECT**; REJECT means the corpus is not used and is extended or replaced before filing (§5.1).

Items are only ever removed; quotas are never recomputed and no tuning item moves to the confirmatory partition.

```
a  acc_s = tuning accuracy of each source; D = {s : acc_s > 0.80}
b  both dated sources in D                        -> REJECT
c  remove every source in D from both partitions
d  if mmlu_cf remains and a dated source is in D:
     k' = total size of the remaining dated source
     if |mmlu_cf tuning| > k'                     -> REJECT (cap cannot
        be met without moving scored items)
     else drop mmlu_cf CONFIRMATORY items with the highest src_hash
        until |mmlu_cf| = min(|mmlu_cf|, k')
e  pooled accuracy of the remaining tuning partition > 0.80
                                                  -> REJECT
f  confirmatory < 300 or tuning < 120             -> REJECT
g  ACCEPT; record the partitions and the weights w_s
```

If only `mmlu_cf` fails, step d does nothing (k is unchanged).

Pooled estimand and source weights. `_cell_split` draws each seed's replay and eval items uniformly from the pooled confirmatory partition, without stratification. The pooled estimand therefore weights each source s by its share of the confirmatory partition, `w_s = c_s / Σ c`. With the cap the three shares are only roughly equal (the sources differ in size after deduplication, and a headroom drop changes them); the **recorded** `w_s` govern the estimand. They are written to the manifest and the filed version; the realised per-seed shares are reported.

§5.2 Per-seed item counts

Confirmatory seeds (0-140) draw replay and eval items only from the confirmatory partition: `n_replay = 40` and `n_eval` items, disjoint within a seed (`_cell_split`). Tuning seeds (1000-1009) draw only from the tuning partition: `n_replay = 40`, `n_eval` = the rest. Seed-level inference is **conditional on the frozen corpus**; it does not generalise to other item sets.

Rule for `n_eval`. Write SD* = 0.039661, the largest SD_D at which N = 141 still has 80 % power for BOUNDED-NO-GAIN / NI at δ = 0.01 (§3). Item sampling contributes about `p_disc / n_eval` to the variance of D, where `p_disc` is the fraction of eval items on which the two post-checkpoints of a contrast differ in correctness. The item part is allowed half of the variance budget: `n_eval = max(130, ceil(p̂_disc / (SD*² / 2)))`, computed with the exact SD* printed by the power script (`SD* / √2 ≈ 0.0280`), where `p̂_disc` is measured on the tuning data (the mean over the 10 tuning seeds, for the two selected configurations) and frozen in the tuning amendment. The planning value `p_disc = 0.10` gives 128, hence the floor of 130; 0.20 gives

255. If the rule needs more than (confirmatory partition size − 40) items, `n_eval` is capped at that value (the frozen corpus is never changed after filing) and the amendment states the reduced power before the first confirmatory seed. This is an approximation (it ignores the finite-population correction and the training noise inside `p_disc`); it fixes the design in advance and uses no confirmatory data.

§5.3 Pilot / smoke set

The pilot fixtures keep their item-disjoint partition (`scripts/e1_fixture_partition.py`, manifest `tests/fixtures/e1_partition_2026-10.json`, `--check` verifies it). They are used only for the pre-filing smoke (§9) and pipeline tests. No tuning or confirmatory cell, and no number in §3, uses them.

§5.4 Profiles and matched compute

Token budget T (independent of any tuned value). For seed s, `T_s = 5 × tok(p_min corpus of seed s)`, where `tok` counts loss tokens (sequence length − 1, pinned tokenizer) and **5 is a fixed constant** (the harness default `inner`), not the tuned `inner`.

Update schedule (all replaying profiles). Given a profile corpus in `_cell_split` order and an inner-repeat count m (default 5, or the tuned value), the update sequence repeats each record m times consecutively, cycles from the start of the corpus when exhausted, and ends with exactly T_s loss tokens. Records with < 2 tokens are skipped.

Each update is one sequence (batch size 1), momentumless SGD, as in `_replay`. **Terminal update:** if the next full record would take the cumulative count past T_s, with r tokens remaining (1 ≤ r), the record is truncated to its first r + 1 input tokens, so it contributes exactly r loss tokens; its loss is the mean over those r positions, like every other update. Achieved loss tokens therefore equal the profile's update budget exactly (T_s, or U_OPR for OPR; see below) for every replaying profile (a unit test covers unequal record lengths and r = 1). Tokens are matched; optimizer-step counts differ across profiles and are reported. DOWNSCALE (`HL_SHRINK = 0.99`, untuned) and RESTRUCTURE are applied after replay and are not charged (they are elementwise / permutation operations with no forward pass). **Scoring passes are counted (PI decision 2026-10-24).** Compute is counted in training-token equivalents: a forward-only scoring token costs 1/3 of a training token (about 2 N vs 6 N FLOPs per token for N active parameters). Pre/post evaluation passes are identical for every profile, so they are counted in every profile's total and cancel in the match. Scoring passes that only some profiles run are charged against those profiles' update tokens: OPR's 160 scoring passes (4 choices × 40 prompts) read S_OPR tokens (prompt + choice per pass), so OPR's update budget is `U_OPR = T_s − ceil(S_OPR / 3)` (floored at 0), and every other profile keeps `U = T_s`. The schedule then ends at U instead of T_s (same terminal-update rule). `scripts/e1_budget.py` (`scoring_pass_tokens`, `update_token_budget`) implements this, with tests; S_OPR, the charge and U are written to each cell. **Feasibility (review round 3).** The match holds only if `ceil(S_OPR / 3) ≤ T_s`. A cell where OPR's scoring alone exceeds T_s is still run (with 0 update tokens) but is classified `scoring−overbudget` (`budget_status`): it is excluded from every matched-compute claim and reported with its counts. A common-budget adjustment (raising every arm's budget by OPR's scoring) was rejected because it would make every arm's T_s depend on OPR and couple the arms. Expected margin: 160 passes of roughly 60-70 tokens give about 3 500 token-equivalents against a T_s of about 12 000 for 40 fact records at m = 5, so the case should not arise; the rule fixes what happens if it does. "Matched compute" therefore means matched total compute (updates plus profile-specific scoring), not only matched update tokens.

Ties. `_score_choice` argmax ties resolve to the lowest choice index (`torch.argmax`); this rule applies to eval and to OPR reward.

Profiles (9 registered + 1 exploratory per confirmatory seed):

- Existing, token-matched: `p_min`, `p_equ`, `p_max`, `p_ctrl_sft`, `p_ctrl_shuffle`, `p_noise` (no updates; the noise floor).
- **`p_ctrl_template` (prompt-only control)** — the same replay items, but each replay text is the **prompt only** (`question + "\nAnswer:"`), with no answer tokens. It trains on the identical wrapper and question text without any answer completion, true or false. It is cycled to T_s. It does not hold completion-format exposure constant, so H14-A is read only as "answer-containing vs prompt-only replay" (§1). (A random-answer control was rejected: three times out of four it teaches a false answer, so beating it could reflect avoiding harmful updates.) A control for prompt artefacts is not a new idea: Zahn et al. (arXiv:2607.16256, v2) ran an in-context analogue, prepending cross-domain material in context to a different, ~671B model. `p_ctrl_template` is a different control: a same-checkpoint, question-without-answer replay arm under the same token budget and update schedule. No novelty is claimed for the control concept.
- **`p_ctrl_vanilla_replay_tuned`** — `p_min`'s corpus with LR and m set by tuning.
- **`p_ctrl_onpolicy_replay` (OPR)** — for each replay prompt, the **pre-replay** checkpoint scores the choices with `_score_choice`; reward = 1 iff its argmax equals the gold answer. The corpus is `prompt + model's argmax choice` for reward-1 items only, cycled to T_s. This is a **restricted multiple-choice adaptation** of Chen et al.'s OPR: no free generation, a binary exact-match reward, and a scoring pass instead of sampling. It is a selection of the checkpoint's own correct choices, not their generated-response method. Claims about OPR in this study are limited to this form. It is also not current-policy replay in the sense of Continual Prompt Replay (arXiv:2608.18574), which regenerates responses with the model being trained: the OPR corpus is fixed once from the pre-replay checkpoint and never regenerated, like every replay corpus in this study (all arms replay frozen responses).

Tuning. Grid LR ∈ {3e-5, 1e-4, 3e-4} × m ∈ {2, 5} (6 configurations), tuning seeds 1000-1009 on the tuning partition. Two independent selections: (i) vanilla replay (`p_min`'s corpus, no DOWNSCALE) and (ii) `p_ctrl_onpolicy_replay`. Each selects the configuration with the highest mean Δ of `acc` over the 10 tuning seeds; ties (within 1e-9) go to the lower LR, then the smaller m. 10 × 6 × 2 = 120 tuning cells (plus the 60 LR-matching calibration cells of the exploratory arm, which select nothing else). **`p_equ` is not tuned separately**: it uses selection (i), so `p_equ` and `p_ctrl_vanilla_replay_tuned` share LR, m and the update sequence and differ only by DOWNSCALE (this is what makes H14-B a DOWNSCALE ablation). OPR's corpus depends only on the pre-replay (base) checkpoint,

so it is the same for every configuration of a tuning seed. A tuning seed whose OPR corpus is empty is dropped, for all configurations alike, from the OPR selection, the SD_D proxy and $\hat{p}$_disc, and is reported; if more than 2 of the 10 tuning seeds are dropped, OPR keeps the default configuration (LR 1e-4, m = 5) and the amendment says so. **PI decision 2026-10-25:** $\hat{p}$_disc and n_eval are computed on the tuning seeds whose OPR corpus is non-empty. If all 10 are empty, H14-C is declared **INFEASIBLE** in the amendment, before any confirmatory seed runs: the OPR arm is not run, H14-C and the secondary contrasts against OPR are reported as INFEASIBLE, and H14-A and H14-B proceed unchanged (the joint headline then cannot be J1). In that case $\hat{p}$_disc is undefined and n_eval takes the §5.2 floor of 130 (the implementation's reading of the decision; to be confirmed by the PI before filing). The two selections, the SD_D proxy, $\hat{p}$_disc and the resulting n_eval (§5.2) are frozen in a dated amendment **before** the first confirmatory seed runs. p_min, p_max and the other controls keep the defaults (LR 1e-4, m = 5); p_equ uses selection

(i) in every confirmatory cell, including secondary contrasts.

- **p_ctrl_lr_matched (exploratory; PI decision 2026-10-24; reported only).** Replay with the learning rate scaled to match the norm change that DOWNSCALE produces (Marek et al., arXiv:2605.26097): p_min's corpus with selection (i)'s m and update sequence and **no DOWNSCALE**, at LR c × LR_(i). c is calibrated once on the tuning seeds 1000-1009 from {1, 1.5, 2, 3, 5}: for each c, the median over tuning seeds of the full-parameter norm change $\|\theta_{post} - \theta_{pre}\|$ is compared with that of p_equ (selection (i) + DOWNSCALE 0.99) on the same seeds; the c with the smallest |log ratio| wins (ties to the smaller c). c is frozen in the tuning amendment with the other selections (60 calibration cells). Contrast Δacc(p_equ) − Δacc(p_ctrl_lr_matched) is reported with the §2 bounds at nominal α = 0.05; it is never confirmatory, outside the family and §7, and never changes a decided state. **PI decision 2026-10-25:** if the calibration is undefined (a median norm change ≤ 0, so the log ratio does not exist), c is recorded as null, the LR-matched arm is not run and the contrast is reported as "not computable"; nothing else changes.

Fixed run knobs: n_replay = 40, n_eval per §5.2, seeds 0-140, tuning seeds 1000-1009, smoke seeds 2000-2002 (pilot set, §5.3, never analysed).

§6 Drift and collapse diagnostics

Per cell: relative weight drift ‖w_post − w_pre‖ / ‖w_pre‖ (over all floating-point parameters), and a **collapse flag** = post acc **<** pre acc − 0.10, **or** post acc below 0.25 (4-way chance). Per profile: collapse rate over seeds.

Frozen-reference monitoring (secondary, descriptive). Motivated by Peng et al. (arXiv:2607.10805) and Khamis et al., *iSDFT: Information-Proximal Self-Distillation for Continual Learning in LLMs* (arXiv:2609.24646), which anchor to the frozen base model. The frozen reference is the pre-replay checkpoint of the same seed (no extra model). Per cell, on a fixed held-out probe set drawn from the confirmatory eval partition's prompts (no gold answers used):

(i) the mean KL divergence from the frozen reference's next-token distribution to the post-replay model's over the 4 answer choices, and (ii) the fraction of items the pre-replay model answered correctly that the post-replay model answers wrongly ("previously correct, now lost"). Both are reported per profile with means and 95 % intervals. They are **not** gates, enter no state rule and do not alter the 20 % collapse gate below; each cell scores a single pre/post replay phase, so collapse over repeated consolidation cycles is not observed; this is a stated limitation.

MoE drift: not applicable. Qwen3.5-4B is a dense model: it has no router and no experts, so the router-drift and expert-usage measures of the E1 draft are recorded as null. Per profile, the mean replay loss over the update sequence is reported with means and 95 % intervals; like the frozen-reference measures, it is not a gate and enters no state rule.

Collapse is an outcome, not an exclusion. **Validity gate:** if the collapse rate of **either** profile in a confirmatory contrast exceeds 20 %, that hypothesis's state is UNINTERPRETABLE (the contrast is reported, but it is not used to rank methods). This gate precedes every substantive state, so a collapsing arm can never produce a SUPERIOR, NON-INFERIOR or BOUNDED reading.

§7 Outcome rules

Each hypothesis is read through its state (§2), so every outcome has a reading:

Hypothesis	State	Reading
H14-A	SUPERIOR	In this experiment, answer-containing replay improves `acc` over prompt-only replay at matched tokens (positive effect, estimate ≥ δ). Answer format and answer content are not separated.
H14-A	POSITIVE-BELOW-SESOI	Positive, with an estimate below δ; reported, no claim.
H14-A	BOUNDED-NO-GAIN	In this experiment, answer-containing replay adds less than δ over prompt-only replay. No statement about the 2026-06-19 mega_v2 residual.
H14-B	SUPERIOR	Adding DOWNSCALE to tuned vanilla replay (same configuration and updates) improves `acc` (positive effect, estimate ≥ δ).
H14-B	POSITIVE-BELOW-SESOI	Positive, with an estimate below δ; reported, no claim.
H14-B	BOUNDED-NO-GAIN	DOWNSCALE adds less than δ to tuned vanilla replay in this setting. Consistent with a REPLAY-only active core at this tier.
H14-C	NON-INFERIOR	`p_equ` is not worse than restricted MC-OPR by δ or more.
H14-C	INFERIOR (exploratory)	Exploratory: `p_equ` appears worse than restricted MC-OPR by more than δ.
any	INCONCLUSIVE	Inconclusive at n = 141 for this contrast; no claim either way.
any	UNINTERPRETABLE	Validity gate failed; reported descriptively only.

DOWNSCALE caveat (H14-B SUPERIOR). DOWNSCALE is a weight shrink (`HL_SHRINK = 0.99`, untuned) applied after replay. Marek et al. (arXiv:2605.26097) show that learning rate and step count directly trade off forgetting against learning, so a shrink of the weights may act as an **implicit learning-rate reduction or regularisation**. An H14-B SUPERIOR reading is therefore compatible with that mechanism and is **not** evidence of a sleep-specific one. Only the registered contrast (tuned replay with versus without DOWNSCALE, same configuration and updates) is tested. The LR-matched replay arm that reproduces the weight-norm change of DOWNSCALE (`p_ctrl_lr_matched`, §5.4) is registered as **exploratory** only (PI decision 2026-10-24): it is reported next to H14-B and cannot change this reading.

Joint headline (first matching row; the last row catches every other combination):

#	Condition	Headline	EC / FC
J1	A = SUPERIOR ∧ B = SUPERIOR ∧ C = NON-INFERIOR	Candidate LLM-tier framework-C signal on this fact corpus: answer-containing replay beats prompt-only replay, DOWNSCALE adds to tuned replay, and the dream arm is not worse than restricted MC-OPR. Needs replication before any EC promotion.	EC stays PARTIAL ; FC unchanged
J2	A = SUPERIOR ∧ B = BOUNDED-NO-GAIN	Answer-containing replay helps; DOWNSCALE adds < δ to tuned replay. No DR-4 support at this tier in this setting.	unchanged
J3	A = BOUNDED-NO-GAIN	In this setting, answer-containing replay adds < δ over prompt-only replay. The mega_v2 candidate is **not** closed by this result; closing it needs a replication on that task.	unchanged
J4	any other combination	No joint headline; per-hypothesis readings only.	unchanged

§8 Reporting commitment and stopping rule

- **Fixed N = 141 confirmatory seeds, no optional stopping, no interim tests.** The run is resumable (JSONL cell cache). Partial caches are not analysed for the confirmatory family.
- If compute ends before 141 complete seeds, the achieved N is reported and **all** H14 hypotheses become exploratory. The run is not re-labelled confirmatory after the fact.

- Each instrument is a different measurement instrument: numbers are never pooled across instruments or studies (the registered CUDA / transformers instrument of §9.2, the `torch-mps` pipeline-smoke instrument of §9.7, the MLX instrument of the E1 draft), and the backend name is part of every cell's run identity; headroom, tuning, calibration, amendment and confirmatory cells must all come from the registered instrument (§9.2), which the harness enforces.
- All per-profile scalars (and per-benchmark scalars of the smoke), token and step counts, drift and collapse diagnostics are reported regardless of outcome.

§9 Substrate, instrument and feasibility (E1-small)

§9.1 Model pin (PI decision S1, decided)

No Qwen3.6 4B model exists on Hugging Face (2026-10-25: the Qwen3.6 releases are `Qwen3.6-35B-A3B` and `Qwen3.6-27B`, with FP8 variants). Two dense 4B candidates were compared:

	`Qwen/Qwen3.5-4B` (pinned)	`Qwen/Qwen3-4B-Instruct-2507` (alternative)
Revision (full sha)	`851bf6e806efd8d0a36b00ddf55e13ccb7b8cd0a`	`cdbee75f17c01a7cc42f958dc650907174a`
Last modified	2026-03-02T00:52:52Z (repo created 2026-02-27)	2025-09-17T06:56:53Z
Generation	Qwen3.5, the architecture family of Qwen3.6 (35B is `Qwen3_5MoeForConditionalGeneration`)	Qwen3 (2025)
Text architecture	dense, 32 layers, hybrid: 3 Gated DeltaNet (linear attention) + 1 gated full-attention layer per block of 4, hidden 2560	dense, 36 full-attention layers, hidden 2560
Post-training	post-trained (hybrid thinking), like the 35B pin	instruct, non-thinking
`tokenizer.json` SHA-256	`5f9e4d49…81cb42`, **identical** to the pinned 35B tokenizer	`aeb13307…92dae4`, different (151 936 vs 248 320 vocabulary)
Licence	Apache-2.0	Apache-2.0

Decision (S1, PI 2026-10-25 19:50): `Qwen/Qwen3.5-4B` at `851bf6e…`. It is the closest sibling of the 35B pin: same model generation and architecture family (minus MoE), same post-training style, and a byte-identical tokenizer, so the corpus build (§5.1b step 3 of the E1 draft), every loss-token count `T_s` and every scoring pass use the same token sequences for both studies. The alternative would change three things at once (tokenizer, attention architecture, post-training generation); it is not used.

Tokenizer comparison (4B pin vs 35B pin). At the two pinned revisions: `tokenizer.json` SHA-256 `5f9e4d4901a92b997e463c1f46055088b6cca5ca61a6522d1b9f64c4bb81cb42` (12 807 982 B) in both; `vocab.json` `ce99b4cb…dfa003` and `merges.txt` `a9d356d7…1db8d` identical (same SHA-256 as the E1 draft §10 item 1, same git blob ids `0aa0ce06…` / `a494e019…`). `tokenizer_config.json` differs (`316230d6…64ac8` vs `5186f0de…2b29b`) **only in `chat_template`** (sorted-key JSON diff: one key); the harness never applies a chat template, adds no BOS (`bos_token` is `None`), and scores raw cloze prompts, so tokenisation is identical.

What is loaded. The checkpoint is multimodal (`Qwen3_5ForConditionalGeneration`: 426 language-model tensors, 297 vision-tower tensors, 15 multi-token-prediction tensors). `AutoModelForCausalLM` (transformers 5.7.0) loads the text model `Qwen3_5ForCausalLM` only: 4 205 751 296 parameters, tied input / output embeddings, layers at `model.layers` (32); the vision tower and the MTP head are not instantiated, as for the 35B harness. This was checked on KXKM-AI on CPU (no GPU) on 2026-10-25.

Weights on the run node. Downloaded on 2026-10-25 with `huggingface_hub.hf_hub_download(revision=851bf6e…)` into `/ai/saisail/e1-small/models` (HF cache layout) on KXKM-AI; only the files below; every SHA-256 recomputed locally and equal to the Hub's LFS SHA-256 / size:

file	bytes	SHA-256
model.safetensors-00001-of-00002.safetensors	5 329 398 688	26a93f066e1916adb13453dae5a0c707c0fbc
model.safetensors-00002-of-00002.safetensors	3 990 429 408	cb544bd9bfae93dc59b0f22b292f593357385
model.safetensors.index.json	76 196	cf3f798ee02ba45f9622aa8892a47369ab667
config.json	3 161	ddc63e1c717afa86c865bb5e01313d89d72bb
tokenizer.json	12 807 982	5f9e4d4901a92b997e463c1f46055088b6cca
tokenizer_config.json	16 710	316230d6a809701f4db5ea8f8fc862bc3a6f3
vocab.json	6 722 759	ce99b4cb2983d118806ce0a8b777a35b093e2
merges.txt	3 353 259	a9d356d7bdf1ef4949e3e748e95b8e10ad9d4
chat_template.jinja	7 756	a4aee8afcf2e0711942cf848899be66016f8d
LICENSE	11 544	bbedc3fda3305820b977265f01b8619d87570

Total 9 342 827 463 B ≈ 9.34 GB (weights 9 319 828 096 B). The harness refuses any other revision of this repository; the runners require the snapshot to hold exactly these ten files and check each SHA-256 before the harness starts (§9.4). The same files, copied to macM1 for the pipeline smoke (§9.7), were checked against the same list.

§9.2 Backend, instrument and kernels

Registered instrument. PyTorch 2.11.0 (CUDA 13.0 wheels) + HF transformers 5.7.0 on one RTX 4090 of KXKM-AI, bf16 parameters, gradients and SGD update, float32 loss and scoring log-softmax, every language-model parameter on that one device. Revision 9 of the E1 draft moved the 35B study to Apple MLX on the Mac Studio (its deviations D1-D10); E1-small keeps the CUDA / transformers path (PI decision of 2026-10-25 14:43). Restated for the 4B, the relevant points of those deviations are: the transformers causal-LM load of this checkpoint is the language model only (§9.1), so full-parameter SGD, DOWNSCALE and the drift / norm measures cover the language model (as `mlx_lm` does for the 35B); transformers stores the RMSNorm weight `w` of `norm(x)·(1 + w)` natively; CUDA replay is not guaranteed bit-reproducible and no claim relies on it; RESTRUCTURE swaps two adjacent entries of the decoder-layer list, masks following `config.layer_types` by position (for unpadded batch-1 sequences the same outputs as per-layer types). The MLX-only D4, D8 and D9 do not apply (their CUDA counterparts: §11 rows 8, 10 and 6). The harness records, in every cell, the backend, the actual parameter placement, the tokenizer kind, the linear-attention implementation and the versions of torch, transformers, tokenizers, fla-core and causal-conv1d, the snapshot and SGD implementation, and accepts E1-small cells only from this instrument (the `torch-mps` instrument of §9.7 only for the smoke).

Tokenizer (S8, decided). Every token id and count of the harness (scoring, `T_s`, `S_0PR`, replay inputs) comes from the raw pinned `tokenizer.json` through the `tokenizers` package, no special tokens: the counter that built the corpus (§5.1b step 3), not the transformers tokenizer (which differs on 1 of 7 850 corpus texts, a Devanagari combining mark, per the E1 draft). The run node pins `tokenizers` 0.22.2 (transformers 5.7.0 requires ≤ 0.23.0); the corpus was built with 0.23.3. `scripts/e1_small_token_ids.py` hashes the ids of every harness text of the corpus (each prompt and prompt + choice, 7 850 texts, at most 158 tokens): both versions give the same digest `41886b68d4737b06010914947689ab9ec75739d25cb425134ec02e298824488c` (2026-10-25; 0.23.3 on macM1, 0.22.2 on KXKM-AI).

Kernels (S6): evaluated, then dropped. The PI first chose (2026-10-25 19:50) to run Gated DeltaNet through `fla-core` 0.5.2 (chunked and fused-recurrent gated delta rule, fused gated RMSNorm) and `causal-conv1d` 1.7.0, the transformers fast path; a model counts as `fla+causal_conv1d` only if every linear-attention layer uses all five fast-path components (chunked and recurrent delta rule and gated RMSNorm from fla, causal conv and its update from causal-conv1d). The torch fallback stays selectable (`HL_LINEAR_ATTN=torch`) but is not the registered instrument. Pins and origins (PyPI):

package	version	artefact	SHA-256
fla-core	0.5.2 (2026-07-27)	wheel py3-none-any	5e830c85bad3d0d34677f9
einops (fla-core dependency)	0.8.2	wheel py3-none-any	54058201ac7087911181bf
causal-conv1d	1.7.0 (2026-08-20)	sdist (no wheel exists for torch 2.11)	3202758494eaa7b597ce1c
causal-conv1d, built	1.7.0	causal_conv1d-1.7.0-cp312-cp312-linux_x86_64.whl, 206 783 377 B	2bc3ec8e091ab063a81960

fla-core 0.5.2 was preferred to 0.6.0 (released 2026-10-23, two days old). causal-conv1d was built on KXKM-AI on 2026-10-25 from the pinned sdist with CAUSAL_CONV1D_FORCE_BUILD=TRUE (no prebuilt download), --no-build-isolation against the run venv's torch, gcc 13.3.0 and a build-only CUDA 13.0 toolchain installed apart from the run venv (scripts/e1_small_cuda_build_requirements.txt: nvidia-cuda-nvcc 13.0.88, nvidia-cuda-cccl 13.0.85, nvidia-cuda-crt 13.0.88, nvidia-nvvm 13.0.88, ninja 1.13.0, all hash-pinned); the built wheel was installed with --require-hashes --no-deps by its hash above.

Kernel parity check (S6; before the measurement smoke). scripts/e1_small_kernel_parity.py loads the 4B with the kernels and then with the torch fallback (one at a time, bf16, raw tokenizer) and compares, on 25 pilot MMLU items (100 prompt + choice sequences) and two replay texts: per-token log-probabilities, _score_choice scores and argmax, the mean replay loss and the full gradient. Tolerances, fixed before the run: max |Δ log p| ≤ 0.25, mean |Δ log p| ≤ 0.02, max |Δ choice score| ≤ 0.05, argmax agreement ≥ 0.95, relative loss difference ≤ 0.01, gradient cosine ≥ 0.99, gradient norm ratio within 1 ± 0.02, relative gradient error ‖g_kernels − g_torch‖ / ‖g_torch‖ ≤ 0.15, and the same set of parameters with a gradient. These are sanity bounds for two summation orders of the same bf16 computation; they do not make the two implementations interchangeable instruments, which is why one implementation is fixed for the whole study. A failure is reported to the PI before any further GPU step.

Result (2026-10-26 23:05, KXKM-AI RTX 4090, code 6b25e48, /ai/saisail/e1-small/out/kernel-parity.json SHA-256 37d9789c…727e4e): **FAIL.** max |Δ log p| 0.340 (> 0.25), mean |Δ log p| 0.0242 (> 0.02), max |Δ choice score| 0.112 (> 0.05) fail; argmax agreement 0.96, relative loss difference 0.0029, gradient cosine 0.99921, norm ratio 1.0053, relative gradient error 0.040 and gradient coverage pass. Peak max_memory_allocated with a full gradient: 20.87 GB (kernels), 21.00 GB (fallback). The bounds are not changed and this failure stands.

Diagnostic (PI decision 2026-10-26 22:09; fixed before the run). scripts/e1_small_kernel_diag.py, same inputs as the parity check (25 pilot MMLU items, 100 prompt + choice sequences and the 2 replay texts, whose tokens join the per-token set), forward passes only, three loads one at a time: bf16 kernels, fp32 torch fallback (the reference), bf16 torch fallback. For each bf16 implementation it reports, against fp32, max and mean |Δ log p| per token, max |Δ choice score| and argmax agreement with fp32. **Rule:** the kernels are the registered instrument only if their mean |Δ log p| to fp32 is ≤ the fallback's **and** their argmax agreement with fp32 is ≥ the fallback's; otherwise the torch fallback is registered and fla / causal-conv1d are dropped (ties go to the kernels, "at least as close on both").

Result (2026-10-26, KXKM-AI, code 154341b; /ai/saisail/e1-small/out/kernel-diag.json SHA-256 d6d1a354…21eb21; 102 sequences, 5 683 scored tokens; a first attempt at 4a13f9c stopped at the script's own dtype check because the fp32 reference had loaded in bf16, and computed no distance):

distance to fp32	bf16 kernels	bf16 torch fallback
max	Δ log p	per token
mean	Δ log p	per token
max	Δ choice score	
argmax agreement with fp32	**1.00** (25/25)	**0.96** (24/25)

The kernels have the better argmax agreement but a larger mean |Δ log p| (by 0.00043), so they are not at least as close on both: **by the rule, the transformers torch fallback is the registered instrument and fla / causal-conv1d are dropped.** They were uninstalled from the run venv on 2026-10-26; the runner sets HL_LINEAR_ATTN=

torch and the harness accepts registered E1-small cells only with linear_attn = torch. The kernels-versus-fallback distances reproduce the failed parity check (max |Δ log p| 0.340, mean 0.0243). Forward-only peak max_memory_allocated: 9.35 GB (bf16), 17.66 GB (fp32).

§9.3 Memory, runtime, headroom and corpus

Memory (arithmetic, bf16, full-parameter momentumless SGD, batch 1). Text parameters P = 4 205 751 296.

- Weights: 2P B = 8.41 GB (7.83 GiB) on the GPU.
- Gradients: every text parameter gets one (dense model; the tied embedding gets a dense gradient): another 7.83 GiB.
- Optimizer state: none (momentumless SGD, foreach in-place update).
- Activations for ≤ 512-token sequences (the §5.1 bound; the longest corpus harness text has 158 tokens, §9.2): 32 layers × hidden 2560 / MLP 9216 stored for backward, the Gated DeltaNet state in float32 (mamba_ssm_dtype), and the 248 320-way logits with the float cross-entropy: about 2-4 GiB.
- CUDA context and allocator slack: about 0.5-1 GiB.

Expected peak ≈ 18-21 GiB of the 23.99 GiB card. Other tenants of KXKM-AI's GPU used about 1.4 GiB on 2026-10-25 (a shared model-swap proxy can load more), so the runner checks free memory before starting (§9.7). **S5 (decided):** only if the measurement smoke shows a peak that does not fit is activation (gradient) checkpointing added, which leaves the update rule unchanged. It is **not implemented** today: if needed, it is implemented, tested and recorded in every cell (a flag in the run identity) before the smoke is repeated and before filing, and it then applies to every cell of the study. A change of optimizer, precision or batch is not allowed without an amendment. The 35B arithmetic ("only routed experts receive gradients") does not apply.

Host. The CPU restore snapshot is the full state_dict copy: 9.68 GB (the tied embedding is stored under two keys). KXKM-AI has 62.6 GiB of RAM (about 42 GiB available on 2026-10-25), an Intel i7-14700KF and 1.8 TB of disk (≈ 700 GB free).

Runtime (estimate; the smoke measures it). Per confirmatory cell at n_eval = 130: 1 040 scoring forwards (pre + post, 4 choices), + 160 for OPR, + about 200 forward / backward updates (T_s ≈ 12 000 loss tokens at m = 5), + one 9.7 GB restore, with the torch fallback for Gated DeltaNet (S6); the measured projection is in §9.7d. The measurement smoke on KXKM-AI (30 cells, §9.7) and its projection to the confirmatory workload replace this estimate in the filed version. If the projection exceeds the available KXKM-AI GPU time, the PI amends before filing; N is never reduced after filing.

One kernel implementation for the whole study. The registered measurement smoke, headroom, tuning, calibration and confirmatory cells all use the transformers torch fallback (§9.2); the harness refuses E1-small cells with another implementation. Switching mid-study is a deviation.

Corpus (same items, model-independent). E1-small uses the **same** PENDING-HEADROOM fact corpus built on 2026-10-25 (corpus file f8379b81…c0a2, confirmatory partition 546f44c6…cb3f, tuning 7aea63a8…b0f7; manifest docs/milestones/e1-corpus-manifest-2026-10-25-pending.json). Its construction depends on the model only through the step-3 token bound (question + gold ≤ 256 tokens), counted with the pinned 35B tokenizer, which is byte-identical to the 4B tokenizer (§9.1); it is therefore the same corpus for both models in every respect.

Headroom (step 9, separate for the 4B; S2, decided). The 4B base checkpoint is scored on the TUNING partition only (HL_STUDY=e1-small, --mode headroom); confirmatory items are never scored before the first confirmatory seed. headroom_rebalance (§5.1b) is applied to the 4B scores exactly as specified (pooled and per-source 0.80 rule, REJECT conditions, weights w_s), with scripts/e1_corpus_build.py --freeze --study e1-small, which accepts only headroom scores of the 4B pin. The result is the E1-small FROZEN corpus and manifest, written to their own directory and hashed into this document; the harness refuses an E1-small run on a corpus frozen with another model's headroom. The 35B study later applies its own step 9 to the same pending corpus.

Consequences of S2. (i) If a source exceeds 0.80 for one model and not the other, the two FROZEN corpora differ (different sources and weights w_s); since nothing is pooled across sizes this does not invalidate either study, but per-source and pooled estimands then differ between the studies, which any descriptive side-by-side must state. (ii) A 4B model is expected to be below 0.80 on all three sources, so the expected E1-small outcome is ACCEPT with no drop.

(iii) The alternative — freezing one corpus for both studies using the stricter of the two headroom results — would need the 35B headroom first, which the PI has deferred; it was not chosen.

Headroom and freeze — done (PI go 2026-10-28 15:45). Headroom: KXKM-AI, registered deterministic instrument, code `bca6170`, 2026-10-28 15:44:57-15:47, TUNING items only (472; no confirmatory item was scored), 152 / 472 correct; result file `docs/milestones/e1-small/headroom-2026-10-28.json` (SHA-256 `b0622633…51302044`). An N10-C measurement job of another agent started on the GPU at 15:46:03, during the last minute of this run; the run is forward-only in deterministic mode and completed normally. Freeze: macM1, `scripts/e1_corpus_build.py --freeze --study e1-small` with the pinned 35B / 4B `tokenizer.json` and `tokenizers` 0.23.3 (the corpus build environment, Python 3.14.4), 152 s:

	realtimeqa	daily_oracle	mmlu_cf	pooled
4B TUNING accuracy	0.393	0.185	0.374	0.322
weight w_s	0.346	0.308	0.346	—

`headroom_rebalance`: **ACCEPT**, no source dropped (every source and the pool ≤ 0.80); the `daily_oracle` accuracy is below the 4-choice chance level 0.25, which the rule does not constrain. **E1-small FROZEN corpus:** file SHA-256 `f8379b810353d25e3c85e4e94b9a8d02553ea8c18abdef631a0289339497c0a2` (byte-identical to the pending corpus: no item was removed), confirmatory partition (1,098 items) `546f44c6d2fc23f7a355a9182dd3e635b47cd520`; tuning partition (472 items) `7aea63a8630eac6c9bf045d0644db898c7b779f35bfb98107114905b35afb0f7`; **FROZEN manifest** (no item text) `docs/milestones/e1-small/corpus-manifest-frozen-2026-10-28.json`, SHA-256 `093fdf610b2c7304a3d8249d2563af777f6ea35ce4379aa008dbc12de77202ab`. These are the hashes every E1-small tuning, calibration and confirmatory cell must load.

§9.4 Code (harness support for the 4B)

- Study selector: one table of pinned models, `scripts/e1_sources.py` `STUDY_MODELS` (`e1` → `Qwen/Qwen3.6-35B-A3B` @ `995ad96e…`, `e1-small` → `Qwen/Qwen3.5-4B` @ `851bf6e8…`), read by the single model-selection spot of `scripts/e1_harness.py` (`MODEL_REPO`, `MODEL_REVISION`) according to `HL_STUDY` (default `e1`; an unknown value is refused). Every model check of the harness core (per-mode refusal, tuning / calibration provenance) and of the PyTorch backend (load at the pinned revision) reads the selected pin, so a cell is accepted only from the pinned model of its study.
- Instruments (`STUDY_INSTRUMENTS`): for `e1-small` the harness accepts
  (a) the registered instrument, backend `torch` with every parameter on one CUDA device, raw tokenizer, torch fallback kernels, RAM snapshot and `torch.optim.SGD`, deterministic mode (every field of `determinism_record()` as registered: S10; S11 adds the allocator setting `expandable_segments: True` to the checked fields), in every mode, and (b) the pipeline-smoke instrument `torch-mps` (every parameter on the MPS device, raw tokenizer, torch fallback kernels, disk snapshot, fused SGD) in smoke mode only. The check applies to the running backend (before the base snapshot is taken), every tuning / calibration cell it reads (which must also carry `study = e1-small`) and the amendment (which records `study` and `backend`). CPU, offloaded or multi-device placements and the MLX backend are refused for E1-small.
- Tokenizer and kernels: `RawTokenizer` (the pinned `tokenizer.json` through `tokenizers`, SHA-256 checked, no special tokens) for E1-small; `HL_LINEAR_ATTN=torch` forces the transformers fallback for models loaded afterwards (selectable, refused for registered E1-small cells); `linear_attn_impl` records what each loaded model actually uses.
- Unified-memory options (MPS smoke): `HL_SNAPSHOT=disk` (base weights as one file per tensor, restored in place: the same values as the RAM snapshot) and `HL_FUSED_SGD=1` (the momentumless SGD step `p -= lr·g` applied per parameter as soon as its gradient is complete, then the gradient freed: the same update as `torch.optim.SGD`, no full-model gradient buffer). Unit tests check identical weights and an exact restore (untied and tied weights). Both are recorded in every cell and are part of the instrument check: the registered CUDA instrument requires the defaults (RAM snapshot, `torch.optim.SGD`).
- Corpus consumer: tuning, calibration, amendment and confirmatory modes refuse a FROZEN manifest whose headroom record does not name the study's pinned model and revision, so E1-small can never run on a corpus frozen with the 35B headroom (and vice versa once the 35B freeze records its model).

- File integrity: `scripts/run_e1_small_cuda.sh` requires the pinned snapshot to hold exactly the ten files of §9.1, checks each against `scripts/e1_small_weights.sha256`, checks that PyTorch sees a CUDA device and runs offline (`HF_HUB_OFFLINE=1`); `scripts/run_e1_small_mps.sh` does the same for the MPS smoke.
- Provenance: each cell and the headroom file record `study`, model and revision (model and revision are part of the run identity).
- Corpus freeze: `scripts/e1_corpus_build.py --freeze --study e1-small` accepts headroom scores from the E1-small pin only; the step-3 tokenizer pin is unchanged.
- Environment on KXKM-AI: `/ai/saisail/e1-small/venv`, CPython 3.12.3, installed with `pip --require-hashes --no-deps --only-binary=:all:` from `scripts/e1_small_cuda_requirements.txt` (57 packages: torch 2.11.0, transformers 5.7.0, huggingface-hub 1.13.0, tokenizers 0.22.2, safetensors 0.7.0, numpy 2.4.4, scipy 1.17.1, pytest 9.0.3, CUDA 13 wheels and their dependencies). Every version **and every hash** is taken from an existing audited lock (the dream-of-kiki `uv.lock` of this branch and the nerve-wml `uv.lock` at `06dd573`, which pins the torch stack). The S6 kernels (fla-core, einops, the causal-conv1d wheel built from its pinned sdist), the only packages that were outside those two baselines, were removed after the diagnostic; their hashes stay recorded in §9.2 and in the requirements file. On macM1 (MPS smoke): `~/scratch/e1-small/venv`, CPython 3.12.13, 38 packages (torch 2.11.0 macOS arm64 and the same transformers / tokenizers / scipy pins), every version and hash from the same two locks.

§9.5 Required pre-filing code tasks (shared with the E1 draft)

1. **Done (PR #42):** paired NI / TOST / bound helper (`kiki_oniric.eval.statistics.paired_tost`, `paired_noninferiority`) implementing §2 exactly, tested against hand-computed examples and SciPy one-sided p-values. The existing `tost_equivalence` is left unchanged and unused here.
2. **Done (2026-10-25):** corpus loader `scripts/e1_corpus_io.py`, wired into `pilot_cycle3_cuda.py --mode`: it verifies the corpus file, every item hash and both partition hashes, and each mode reads one partition only (confirmatory: CONFIRMATORY of a FROZEN corpus; tuning / calibrate: TUNING of a FROZEN corpus; headroom: TUNING of the PENDING-HEADROOM corpus; smoke: the pilot fixtures through their partition manifest). Seeds are fixed per mode (0-140, 1000-1009, 2000-2002); the pinned model revision is enforced at load.
3. **Done (2026-10-25):** token-budget schedule `scripts/e1_schedule.py` (T_s, m-fold repeats, cycling, terminal truncation to exactly the budget), wired with the §5.4 scoring charge of `scripts/e1_budget.py`; achieved tokens, optimizer steps and truncations are written to each cell.
4. **Done (2026-10-25):** profile table, corpora (prompt-only template, restricted MC-OPR with corpus size and S_OPR per cell), tuning selection, empty-OPR handling, LR-matching calibration of c and the frozen tuning amendment (`scripts/e1_profiles.py`, `--mode tuning|calibrate|amend`).
5. **Done (2026-10-25):** per-cell drift (full norm), collapse flag, frozen-reference KL and "previously correct, now lost", router drift, expert-usage L1 and mean replay loss (router drift and expert usage are null for the dense 4B, §6) (`scripts/e1_diagnostics.py`; operationalisation in its docstring: probe set = the cell's own eval items, KL over the softmax of the 4 choice scores).
6. **Done (2026-10-25):** `analyze_e1_cuda.py` replaced by the §2 / §4 / §6 / §7 decision code (states, gates, joint headline, secondary and exploratory contrasts, per-source description).
7. **Done (2026-10-25)** up to step 8: `scripts/e1_fetch_sources.sh`, `scripts/e1_sources.py` and `scripts/e1_corpus_build.py` (pins, row mapping, rules 1-9, the reference core, manifest without item text, `--check`, `--freeze` for step 9). Step 9 needs the base checkpoint on the CUDA node (`--mode headroom`, TUNING only). All code is torch-free and tested, except the harness wiring, whose tests run on a tiny CPU model where torch is installed.
8. **E1-small (2026-10-25):** study selection, instruments, raw tokenizer, kernel selection, MPS smoke path, per-study freeze, the kernel parity and token-id checks (§9.2, §9.4), tested in `tests/unit/scripts/test_e1_study.py`, `test_e1_small_torch.py`, `test_e1_small_tools.py` and `test_e1_corpus_build.py`.

§9.6 Corpus build record (the shared pending corpus)

Corpus build, steps 1-8 (2026-10-25, PI go). Code at `d82a512` (build path unchanged since), host macM1 (M1, 32 GB, macOS 27, CPU, `nice` 19), Python 3.14.4, tokenizers 0.23.3, pinned tokenizer of the E1 draft §10 item 1 (byte-identical to the 4B tokenizer, §9.1); sources fetched on macM1 with `scripts/e1_fetch_sources.sh` and

verified against their pins (Daily Oracle and MMLU-CF SHA-256 equal to §5.1a; RealTime QA's 41 `past/2026/*_qa.jsonl` files equal to the blobs of commit `e288dcf`; every raw SHA-256 in the manifest). Wall time 159 s, peak RSS 0.6 GB. `--check` rebuilt it bit-identically. Status **PENDING-HEADROOM**: for E1-small, step 9 is the 4B headroom on the CUDA node (`HL_STUDY=e1-small --mode headroom`, TUNING only), then `--freeze --study e1-small` (§9.3). No item was scored.

step (items)	realtimeqa	daily_oracle	mmlu_cf
1 load	700	18 557	10 000
3 exclude	700	18 556 (1 empty field)	4 653 (5 346 rule 3, 1 > 256 tokens)
4 date	545	492	4 653
4 contamination	545	491	4 653
5 dedupe / skeleton cap	543	484	4 544
6 cap (k = 543)	543	484	543
7 tuning / confirmatory	163 / 380	146 / 338	163 / 380

Pooled: **confirmatory 1 098** (≥ 300), **tuning 472** (≥ 120); pre-headroom weights w = 0.346 / 0.308 / 0.346 (`realtimeqa` / `daily_oracle` / `mmlu_cf`). No item fell in a component with a reference row; the skeleton cap removed none. Rule 3 removals per variant (items carrying it): `None of the other choices` 4 634, `None of the mentioned` 178, `None of these` 141, `None of the above` 137, `none of the mentioned` 87, `none of these` 51, `none of the above` 42, `None of the above.` 15, `None of the Mentioned` 14, `none of the above.` 10, `none of these.` 7, `None of these.` 5, `None of the above is correct.` 4, `None of These` 3, `None of above` 3, `none of the choices.` 3, `None is applicable` 2, `None of these alternatives is correct.` 2, and 1 each for the other nine strings (an item can carry two). Gold option longest (descriptive, rule 2): 199 / 543, 165 / 484, 212 / 543. Corpus file SHA-256 `f8379b810353d25e3c85e4e94b9a8d02553ea8c18abdef631a0289339497c0a2` (not committed, rule 9); confirmatory partition `546f44c6d2fc23f7a355a9182dd3e635b47cd52082d0b1ac347abbcf4914cb3f`, tuning `7aea63a8630eac6c9bf045d0644db898c7b779f35bfb98107114905b35afb0f7`; manifest (no item text) committed as `docs/milestones/e1-corpus-manifest-2026-10-25-pending.json` (SHA-256 `95c69035…d056344`; rebuilt at `e4d85e8` to add the provenance dates of the PI decision of 2026-10-25, corpus file byte-identical). These hashes are pre-headroom; the filed hashes are those of the FROZEN corpus after step 9.

§9.7 Smokes (two roles; PI decision of 2026-10-25 20:06)

Both smokes use the §5.3 pilot fixtures only, seeds 2000-2002, all 10 profiles (30 cells), `--smoke-n-eval 50`, the defaults standing in for the untuned arms (placeholder amendment). Neither is ever analysed and no number of either enters §3 or any state.

1. **Pipeline smoke on macM1 (Apple MPS).** M1 Max, 32 GB unified memory, no CUDA: PyTorch MPS, every parameter on the MPS device, bf16, transformers torch fallback for Gated DeltaNet (fla / causal-conv1d are CUDA-only), raw pinned tokenizer; base snapshot on disk and the SGD step applied per parameter inside backward (`HL_SNAPSHOT=disk`, `HL_FUSED_SGD=1`; same update arithmetic, §9.4). Cells carry backend `torch-mps`; the harness accepts this instrument for the smoke only. Role: run every profile and code path end to end on the real 4B (restore, budgets, OPR, DOWNSCALE, RESTRUCTURE, diagnostics, invariants) before the CUDA node is free. It does not give the runtime projection. Driver: day model paused for the duration (pause always lifted), refused below 40 % free memory, stopped on memory-pressure warning; one probe cell (`p_max`, seed 2000) first.
2. **Measurement smoke on KXKM-AI (registered instrument).** The same 30 cells on the RTX 4090 with the registered instrument. It records per-cell wall time, `torch.cuda.max_memory_allocated`, host RSS, achieved tokens and update counts, and its projection to the 1,620-cell workload (scaling each component by the confirmatory workload, E1 draft method) is written into the filed version. It runs after the kernel parity check (§9.2) and only when the PI releases the GPU.

Results of the macM1 smoke (§9.7a):

§9.7a First attempt, 2026-10-25 20:28-20:32 — stopped, no cell completed. Code 62617e6, weights copied from KXKM-AI and checked against `scripts/e1_small_weights.sha256` (10/10). Day model paused at 20:27:57 (oMLX backend on :8081 stopped), free memory 91 % after the pause. The probe cell (`p_max`, seed 2000) loaded the model on MPS and wrote the 9.0 GB disk snapshot; at 20:32:18, still inside the first cell, the memory-pressure level reached 2 (warn, free 30 %) and the driver stopped as required. Two other jobs were running on macM1 at that time (a bouba v0.8 capture started at 20:14, which the driver's wait pattern did not match, and an E1 MLX parity run started at about 20:30), so the share of the 4B smoke is not known. The driver lifted the pause (:8080 answered HTTP 200 at 20:32:25), but its kill did not reach the harness process, which ran until it was killed by hand at about 20:37. The driver (`scripts/e1_small_macm1_smoke_driver.sh`) now waits for any heavy job (and stops if one starts), stops the harness (TERM, then KILL, the whole launcher process group and the harness) and lifts the pause only once no harness or launcher process is left; a signal ends the driver through the same cleanup. No number from this attempt is used.

§9.7b KXKM-AI measurement smoke, non-deterministic — SUPERSEDED by §9.7d for every measurement and the projection (S10); kept as run (2026-10-26 23:15 → 2026-10-27 00:59; registered instrument after the S6 diagnostic: torch, one CUDA device, raw tokenizer, torch fallback, RAM snapshot, `torch.optim.SGD`; code `cce9ba7`; torch 2.11.0+cu130, transformers 5.7.0, tokenizers 0.22.2). Probe cell (`p_max`, seed 2000): 162.2 s, peak `max_memory_allocated` 19.52 GB, host RSS 13.97 GB, 400 updates, 18 335 / 18 335 loss tokens, no invariant violation. Then the 30 smoke cells (seeds 2000-2002 × 10 profiles, pilot fixtures, 80 eval items per cell, placeholder amendment), cell cache SHA-256 `bf719418…cfab3e`: **30 / 30 completed, 0 invariant violations, every budget matched** (achieved = planned loss tokens), peak `max_memory_allocated` ≤ 20.39 GB (18.99 GiB) of 23.99 GiB, host RSS ≤ 13.99 GB, total wall 1.73 h.

profile	wall per cell (s)	updates	peak (GB)
`p_min`, `p_equ`, `p_max`, `p_ctrl_vanilla_replay_tuned`, `p_ctrl_lr_matched`	163-165	400	20.36-20.39
`p_ctrl_shuffle`	165-167	403-405	20.35
`p_ctrl_template`	205-216	539-582	20.06
`p_ctrl_onpolicy_replay`	159-175	310-365 (charged budget)	19.50
`p_ctrl_sft`	581-779	1 871-2 567 (short texts)	19.50
`p_noise`	46.5-47.0	0	16.16

Two cells carry the collapse flag (`p_ctrl_template` seed 2002, post − pre = −0.113; `p_ctrl_vanilla_replay_tuned` seed 2000, −0.147); the smoke is never analysed.

Run-to-run variability (finding for the PI). In the smoke the untuned arms all run the default configuration, so `p_min`, `p_ctrl_vanilla_replay_tuned` and `p_ctrl_lr_matched` (placeholder c = 1) are the same computation on the same seed, yet their Δ differ: seed 2000 +0.067 / −0.147 / +0.027, seed 2001 +0.110 / +0.060 / +0.050, seed 2002 +0.070 / +0.053 / +0.060; the probe and the smoke `p_max` seed-2000 cells also differ (post 0.577 vs 0.593). CUDA replay is not bit-reproducible (most likely non-deterministic reductions in backward; not isolated here), and 400 SGD steps can amplify it. This is noise within an arm that §3 does not model separately (it enters SD_D); whether to enforce deterministic kernels (`torch.use_deterministic_algorithms`, cuBLAS workspace) before filing is a PI decision (S10, §10).

Projection to the 1,620 cells (`scripts/e1_small_projection.py`): least-squares fit over the 30 cells, wall = 0.0655 s × scoring forwards + 0.2835 s × updates + 0.00020 s × loss tokens + 5.0 s; the fit reproduces the measured cells (e.g. `p_min` 164 s, `p_ctrl_sft` 637 s predicted). Workload: T_s = 5 × 40 × 39.2 mean loss tokens of a corpus fact text = 7 847, updates per profile scaled from the smoke by T_s; confirmatory `n_eval` 130 (1 040 scoring forwards, + 160 for OPR); tuning and calibration on the 432 remaining TUNING items.

n_eval	confirmatory 1,410	tuning 120	calibration 60	smoke 30 (measured)	**total**
130 (floor)	52.5 h (134 s per cell)	9.4 h	4.6 h	1.7 h	**68.2 h**
255 (§5.2 at p_disc 0.20)	78.1 h (199 s per cell)	9.4 h	4.6 h	1.7 h	**93.8 h**

About 3-4 days of the RTX 4090; scoring forwards of the corpus items are assumed to cost as much as those of the pilot items (both ≤ 160 tokens). Memory fits without activation checkpointing (S5 not triggered).

§9.7c Determinism check (S10; PI decision 2026-10-27; 2026-10-27 08:12-08:26, KXKM-AI, code `75b27f0`). Deterministic mode (`HL_DETERMINISTIC=1`): `torch.use_deterministic_algorithms(True, warn_only=False)`, `CUBLAS_WORKSPACE_CONFIG=:4096:8` (required exactly, set before the process starts), cuDNN deterministic on and benchmark off, Python / NumPy / torch CPU and CUDA RNGs reseeded to 0 at the start of every cell; all of it is recorded in every cell's `versions`. `scripts/e1_small_determinism_check.py` ran, with one model load, the smoke-setting cells `p_min`, `p_ctrl_vanilla_replay_tuned`, `p_ctrl_lr_matched` (the same computation under the placeholder amendment) and `p_min` again, all on seed 2000, restoring the base weights between cells (result file SHA-256 `26119d1f…da913b7`):

- **PASS:** the four cells are identical in pre accuracy (0.5433), post accuracy (0.5767), every per-item choice score (pre and post; one scores digest `e2d63a21…`) and the SHA-256 of all parameters after the cell (`adc47906…eafa9845`, different from the base `c1b469d8…`); each ran 400 updates with all 18 335 planned loss tokens and no invariant violation. The restore path therefore returns the cell to the same state every time.
- **No op lacks a deterministic implementation** on this path (no error, no warning; `warn_only=False`).
- **Cost:** 195.4-196.2 s per cell against 163.6 s for the same non-deterministic smoke cell (`p_min`, seed 2000; §9.7b), i.e. about **+20 %** wall; peak memory unchanged (19.55-19.57 GB). Scaling the §9.7b projection by 1.2 gives about **82 h** at `n_eval` 130 and **113 h** at `n_eval` 255 (an estimate: only this cell type was timed).
- The non-deterministic `p_min` seed-2000 smoke cell had post 0.610; the deterministic one has 0.577: the earlier spread is run-to-run noise of the non-deterministic kernels, not a restore defect.

S10 decision (PI, 2026-10-27): registered. The proposal was to register deterministic mode for every E1-small CUDA cell (headroom, smoke, tuning, calibration, confirmatory), enforced by the instrument check, at a cost of about +20 % wall (≈ 82-113 h in total). Reason: without it, identical computations differ by up to 0.21 in Δ, twenty times δ, which inflates SD_D and makes no cell reproducible; with it, a cell is bit-identical on rerun. The PI agreed: the measurement smoke is repeated in deterministic mode (§9.7d) and its projection replaces §9.7b.

§9.7d KXKM-AI measurement smoke in deterministic mode (registered instrument; replaces §9.7b).

First attempt (2026-10-27 08:38-10:10, code `46d7f95`): stopped. The probe cell and 19 smoke cells completed; cell 20 (`p_ctrl_template`, seed 2001) ran out of CUDA memory in backward with 17.0 GiB allocated and 3.5 GiB reserved but unallocated (allocator fragmentation; only ComfyUI's 0.65 GiB besides). Fix (`2384e19`): the runner sets `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`, an allocator setting that changes memory allocation, not the update arithmetic (no numerical difference was observed in the 19 cells shared by both runs, below), recorded in every cell's determinism record (validated by the PI as part of the registered instrument, S11). No number of this attempt is used, except as the cross-run check below.

Rerun (2026-10-27 12:52-14:59, code `2384e19`, tmux `e1s-smoke2`, cell cache SHA-256 `32576f21…e0696f96`). Probe (`p_max`, seed 2000): 198.1 s, post 0.593, identical to the first attempt's probe. Then 30 / 30 cells completed: **0 invariant violations, every budget matched, peak `max_memory_allocated` ≤ 20.08 GB (18.7 GiB), host RSS ≤ 14.0 GB, total wall 2.05 h**. Every cell records the full determinism record and the registered instrument (torch fallback, raw tokenizer, RAM snapshot, `torch.optim.SGD`). One cell carries the collapse flag (`p_ctrl_shuffle` seed 2002, −0.103); the smoke is never analysed.

profile	wall per cell (s)	updates	peak (GB)
p_min, p_equ, p_max, p_ctrl_vanilla_replay_tuned, p_ctrl_lr_matched	194-197	400	20.08
p_ctrl_shuffle	196-197	403-405	20.03
p_ctrl_onpolicy_replay	190-209	310-365	19.43
p_ctrl_template	242-256	539-582	19.92
p_ctrl_sft	686-918	1 871-2 567	19.43
p_noise	56.8-57.3	0	16.11

Determinism, at scale. p_min, p_ctrl_vanilla_replay_tuned and p_ctrl_lr_matched (the same computation) give identical Δ on every seed (+0.033, +0.083, +0.060), and the 19 cells common to the first attempt and the rerun, in two processes, are identical in pre and post accuracy, every per-item correctness, the frozen-reference KL, the drift and the mean replay loss (19 / 19).

Projection to the 1,620 cells (replaces §9.7b). Same method (scripts/e1_small_projection.py, result SHA-256 82a59f14…a53c3655): wall = 0.0814 s × scoring forwards + 0.3335 s × updates + 0.00022 s × loss tokens + 5.1 s; T_s = 7 847.

n_eval	confirmatory 1,410	tuning 120	calibration 60	smoke 30 (measured)	**total**
130 (floor)	63.2 h (161 s per cell)	11.6 h	5.6 h	2.1 h	**82.4 h**
255 (§5.2 at p_disc 0.20)	95.0 h (243 s per cell)	11.6 h	5.6 h	2.1 h	**114.3 h**

About 3.5-5 days of the RTX 4090. Memory fits without activation checkpointing (S5 not triggered).

§9.8 Filing gate (E1-small)

Not filed until: the measurement smoke on KXKM-AI has run on the registered instrument, deterministic mode included (done, §9.7d); the 4B headroom has run and the E1-small FROZEN corpus and manifest hashes are written here (done, §9.3); the decision code has been run on synthetic paired data covering every §2 state and §7 row; the harness code is merged at a pinned commit with passing tests. If the projection exceeds the available KXKM-AI GPU time, the PI amends before filing; N is never reduced after filing.

§9.9 Audit trail

Pre-run records: docs/milestones/e1-small/ (kernel parity and fp32 diagnostic, determinism check, both smokes, projection, headroom, FROZEN manifest). Milestones: docs/milestones/e1-small-4b-replay-controls-<date>.{json,md}, including the frozen tuning amendment, the SD_D proxy, the HF revision, the instrument record of every cell (backend, device placement, tokenizer, linear-attention implementation and kernel versions) and both smokes. Deviations: an amendment commit before the affected seeds run, or post-hoc disclosure marking the affected hypotheses exploratory (per docs/osf-preregistration-draft.md §5).

§10 PI decisions

Decided by the PI on 2026-10-25 (19:50 and 20:06); changing one requires editing the named section before the lock commit.

- **S1 Model — decided:** Qwen/Qwen3.5-4B @ 851bf6e8… (§9.1).

- **S2 Headroom / freeze — decided:** E1-small freezes its own copy of the pending corpus with the 4B headroom, TUNING only (§9.3).
- **S3 Order:** E1-small is filed and run before the 35B study; the E1 draft is not edited here (a one-line cross-reference can be added by its author).
- **S4 Hardware:** KXKM-AI, one RTX 4090, shared with other studies; the GPU is used only when the PI releases it.
- **S5 Memory fallback — decided:** activation checkpointing is implemented only if the measurement smoke shows that memory does not fit; it would then be tested, recorded in every cell and applied to the whole study before filing (§9.3).
- **S6 Kernels — decided:** fla-core 0.5.2 + causal-conv1d 1.7.0 were installed and checked against the torch fallback (2026-10-25 19:50); the parity check failed, and by the fp32 diagnostic rule fixed by the PI on 2026-10-26 22:09 the **torch fallback is registered** and the kernels are dropped (§9.2). The implementation is fixed for the whole study.
- **S7 Text — decided:** this document is self-contained (§1-§8 copy revision 9 of the E1 draft with the edits of §11).
- **S8 Tokenizer — decided:** token ids and counts come from the raw pinned `tokenizer.json` through the `tokenizers` package, no special tokens (the counter that built the corpus); §9.2 records the version check.
- **S9 macM1 pipeline smoke — decided (20:06):** run in addition to the KXKM-AI measurement smoke, on the `torch-mps` instrument (§9.7).
- **S10 Determinism — decided (PI, 2026-10-27):** deterministic mode is registered for every E1-small CUDA cell and enforced by the instrument check (§9.7c); the measurement smoke is repeated in this mode and its projection replaces the superseded one (§9.7b, §9.7d).
- **S11 Allocator — decided (PI, 2026-10-27 15:57):** `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` is part of the registered CUDA instrument (it changes memory allocation, not the update arithmetic; no numerical difference was observed in the 19 overlapping smoke cells; it removed the fragmentation out-of-memory stop of §9.7d). The instrument check refuses a running backend, an imported tuning / calibration cell or an amendment without it, like the determinism fields.

§11 Differences from the E1 (35B) draft (revision 9)

Every place where §1-§9 differ from revision 9 of the E1 draft. In §1-§8 the only text edits are rows 1, 7, 11, 16 and 17 (the §1 differentiation and scope sentences, the §3 “at 4B” sentence, the §5.1a cutoff sentence, the §6 MoE paragraph and the §8 instrument bullet); §9-§10 are E1-small’s own.

#	Item	E1 (35B) draft	E1-small	Forced by
1	Model	`Qwen/Qwen3.6-35B-A3B` @ `995ad96e…`, MoE 35B total / 3B active	`Qwen/Qwen3.5-4B` @ `851bf6e8…`, dense 4.21B (text) (§9.1)	PI decision S1
2	Tokenizer file	`tokenizer.json` `5f9e4d49…`	the identical file (§9.1)	—
3	Harness tokenization	transformers tokenizer (open item of the E1 draft §10)	raw pinned `tokenizer.json` through `tokenizers`, no special tokens (§9.2)	PI decision S8 (same rule announced for the 35B study)
4	Hardware	Mac Studio M3 Ultra, 512 GB unified memory	1 × RTX 4090 24 GB (KXKM-AI), whole model on one device, no device map (§9.2)	PI decision, size
5	Memory arithmetic	≈ 280 GB upper bound in unified memory	≈ 18-21 GiB GPU peak plus a 9.7 GB host snapshot (§9.3)	size, backend
6	Restore per cell	re-binds the immutable base arrays (35B D9)	`load_state_dict` copy of ≈ 9.7 GB host → device	backend

#	Item	E1 (35B) draft	E1-small	Forced by
7	MoE drift (§6)	router drift, expert-usage L1	not applicable, recorded as null; every other §6 measure unchanged	dense model
8	Routed-expert gradients (35B D4)	dense stacked-expert gradient, zero rows for unrouted experts	not applicable: every language-model parameter receives a gradient	dense model
9	RESTRUCTURE (p_max, 35B D7)	swap of two adjacent decoder-layer entries; MLX masks by each layer's type	same swap; transformers masks by position; adjacent layers may differ in type (linear vs full attention)	backend; hybrid layers
10	Memory recording (35B D8)	`mx.get_peak_memory`	`torch.cuda.max_memory_allocated` per device, host RSS	backend
11	Date cutoff rationale (§5.1a rule 1)	2026-04-16, after the 35B release (2026-04-15)	the same cutoff; it also post-dates the 4B release (2026-02-27)	—
12	Weight verification	40 files of `995ad96` (`scripts/e1_model_pins.json`)	the 10 downloaded files of `851bf6e8`, exact inventory (§9.1, §9.4)	model, backend
13	Headroom and freeze (§5.1 property 5, §5.1b step 9)	35B base on TUNING	4B base on TUNING only; its own FROZEN copy of the same pending corpus (§9.3)	PI decision S2
14	Backend	Apple MLX (CUDA kept as a parity path)	CUDA, PyTorch 2.11.0, transformers 5.7.0, hash-pinned (§9.2, §9.4)	PI decision
15	Linear-attention kernels	`mlx_lm` Metal kernel / op-by-op recurrence	transformers torch fallback; fla-core + causal-conv1d evaluated and dropped by the fp32 diagnostic (§9.2)	PI decision S6
15b	Determinism	not specified (MLX replay not bit-reproducible, 35B D6)	deterministic mode registered for every CUDA cell, enforced (§9.7c)	PI decision S10
15c	CUDA allocator	not specified	`expandable_segments: True`, enforced (§9.7d)	PI decision S11
16	Scope (§1)	Qwen3.6-35B-A3B	Qwen3.5-4B; no inference about 35B (§0)	scope
17	Instruments (§8)	MLX registered	CUDA registered; `torch-mps` for the macM1 pipeline smoke only; never pooled (§9.7)	PI decisions
18	Smokes	one smoke on the filing hardware	two: macM1 pipeline smoke (`torch-mps`) and KXKM-AI measurement smoke (§9.7)	PI decision S9
19	Runtime	unmeasured at 35B	estimate (§9.3), measured by the KXKM-AI smoke	size, hardware
20	Entry point	`scripts/e1_mlx.py`	`scripts/run_e1_small_cuda.sh` (registered), `scripts/run_e1_small_mps.sh` (pipeline smoke)	backend
21	Milestones	`docs/milestones/e1-35b-replay-controls-<date>`	`docs/milestones/e1-small-4b-replay-controls-<date>` (§9.9)	separate study

#	Item	E1 (35B) draft	E1-small	Forced by
22	α family	its own 0.05	its own 0.05, no shared correction (§0)	separate study
23	Text	the E1 draft	self-contained copy of §1-§8 (header)	PI decision S7
24	Cell counts	1,620 (incl. 30 smoke)	the same 1,620 (plus the 30 macM1 smoke cells, never analysed)	—

Everything not listed is identical: hypotheses, endpoint, δ, paired procedures, α, power and N, exclusions, corpus construction and partitions, `n_eval` rule, profiles, budgets, scoring charge, tuning grid and selections, LR-matched arm, diagnostics, gates, outcome rules, joint headline, reporting and stopping rule, seeds.